Chapter 1

INTRODUCTION

1.1 AIMS OF MODELING

In general, a *model* is an artificially constructed object that makes the observation of another object easier. To make the obvervation possible, *physical models* of three-dimensional physical things such as buildings, ships, and cars usually share the relative dimensions and general appearance of their physical counterpart, but not the size. *Molecule models* used in chemistry share the relative arrangement of the various atoms of the molecule with respect to each other, but very few of its other properties. *Mathematical models*, widely used in various fields of science and engineering, represent some of the behavioral aspects of the phenomenon modeled in terms of numerical data and equations.

Engineering drawings, widely used in design and engineering, can also be viewed as models. Through a collection of two-dimensional images, aided by conventional symbolism for representing dimensions, tolerances, surface characteristics, materials, and so on, they are capable of conveying true three-dimensional information to an experienced reader.

Models are useful because often one can study certain characteristics of an object more easily based on the model than its physical counterpart. This may be so because the object does not yet exist (as often is the case for physical models), or because it is not directly observable (as is the case for molecules), or because it cannot be examined in a controlled fashion (as often is the case for mathematical models). Note, however, that physical and mathematical models are limited in the scope of their usefulness: a problem of a new type often requires a new model.

As models, engineering drawings try to avoid these problems. They are relatively *general-purpose*: a drawing can be used to extract information for

many tasks, including the production of physical and mathematical models when they are needed. From this universality it follows that they can act as a *medium of communication* between the people that participate in the design of an object. Unlike physical and mathematical models, drawings also support the iterative, "sketchy" nature of design. They can be fuzzy and inaccurate when appropriate, whereas physical and mathematical models are tools for representing or analyzing a relatively finished object.

1.2 COMPUTER MODELS

Computer models consist of data stored in computer files that can be used to perform similar tasks as the other kinds of models discussed above, with the aim of sharing their merits and avoiding their problems. In particular, we aim at general-purpose models that can support a wide variety of applications, just like engineering drawings.

1.2.1 Geometric Modeling

The totality of data that would be stored in a computer model depends on the scope of questions one wants to be able to answer, and *a priori* it does not seem possible to limit the amount of potentially interesting data. At first sight, this may appear to make computer models unattractive.

The key observation that motivates this book is that many problems that we try to solve through models are inherently *geometric*. For instance, the problem of calculating a shaded image of an object includes geometric problems such as the following:

1. Which parts of the object are visible to the viewer?
2. What color should be assigned to each element of the image?

Similarly, the problem of generating control sequences for a numerically controlled machine tool is basically geometric: which sequence of movements of the tool will produce the shape of the object?

If we can represent the geometric shape of the object adequately for these questions, we would be able to provide answers to many others as well. In fact, it appears that the geometry of an object is the most useful part of the bulk of potential information. Moreover, techniques for storing and processing geometric data are relatively independent of particular applications: essentially identical methods can be used to construct models of machines, ships, chemical plants, and baby food containers.

Accordingly, it makes sense to separate the data dealing with the geometric shape of an object from other, nongeometric data. In this approach,

the totality of data needed for a particular scope of problems is called the *object model*, while the purely geometric part of it constitutes a *geometric model*. A geometric model, of course, is a subset of the object model.

1.2.2 Solid Modeling

Solid modeling is a branch of geometric modeling that emphasizes the general applicability of models, and insists on creating only "complete" representations of physical solid objects, i.e. representations that are adequate for answering arbitrary geometric questions *algorithmically* (without the help of interaction with a human user).

The goal of relatively general applicability separates solid modeling from other types of geometric models which are biased towards some special purposes. *Graphical models* are intended for describing a *drawing* of an object rather than the object itself. *Shape models* represent an *image* of an object. They may be unstructured collections of image elements, or may have some internal structure to aid, say, image processing operations. *Surface models* give detailed information on a curved surface, but do not always give sufficient information for determining all geometric properties of the object bounded by the surface.

Because of our interest in general-purpose modeling, we exclude these other types of computer models from the scope of this book. Nevertheless, as we shall see, some methods developed for the construction of these models have found their way to solid modeling as well.

1.3 PROBLEMS OF SOLID MODELING

The requirement of general applicability demands much of the completeness and accuracy of solid models. In the following, we discuss on an intuitive level some of the problems we shall deal with in later parts of this book; in Chapter 3 we shall take a more systematic look into them.

1.3.1 Completeness

The simplest kinds of computer models of physical objects transfer the methods used in engineering drawings directly to a computer to create *two-dimensional graphical models*. These models consist of two-dimensional graphical objects such as lines, arcs, text, and other notation needed in the figure.

Graphical models are perfectly appropriate to make the generation of technical drawings more efficient and to enhance their quality. Unfortunately, whereas human beings (or at least engineers) are capable of inter-

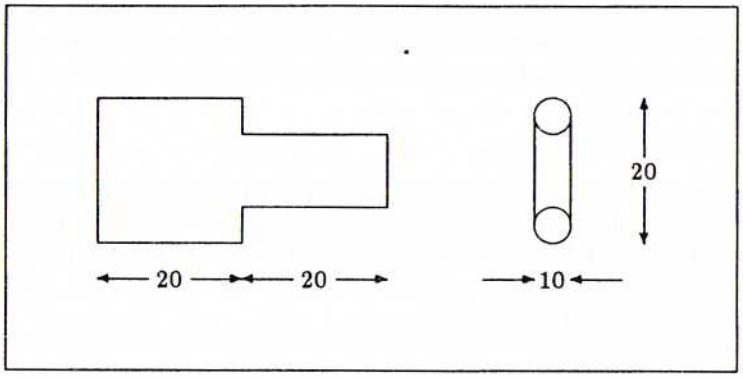

Figure 1.1 A nonsense drawing.

preting engineering drawings properly, computers in general lag far behind them in that skill. Thus graphical models in general cannot serve as solid models.

One of the problems of graphical models is that it is perfectly possible to make drawings that represent no actual shape; see, for instance, Figure 1.1 [105]. Still worse, the problem of algorithmically inferring correct three-dimensional information from a collection of two-dimensional figures remains unsolved for practical purposes, despite some interesting results gained with simplified problems.

With the aid of some computer power, two-dimensional graphical models can be upgraded to their three-dimensional counterparts by adding the information of the third coordinate. This results in a solid representation commonly referred to as the *wire frame* model. With wire frames, it becomes possible to store only a single three-dimensional model and generate all needed two-dimensional views from it. Hence one of the problems of two-dimensional graphical models can be overcome, because it is less easy to make drawings whose views are inconsistent.

Unfortunately, even a collection of three-dimensional lines is not sufficient for representing a shape, because some collections of lines may have several interpretations in terms of solid objects. The standard example is given in Figure 1.2.

The problem of graphical models is that they do not offer *complete* geometric information of the shape of the object—information that would be sufficient for calculating answers to arbitrary geometric questions. Nev-

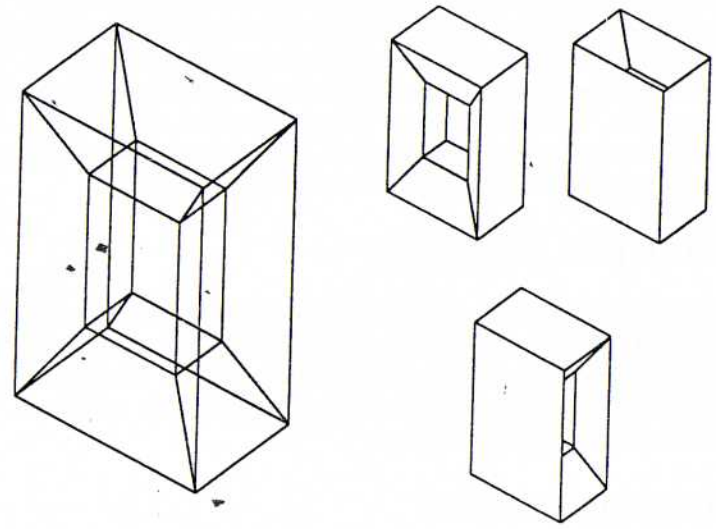

Figure 1.2 An ambiguous graphical model.

ertheless, graphical models are perfectly all right for the task they were originally developed for, namely generation and representation of drawings. We shall study them further in Chapter 2.

1.3.2 Integrity

To solve the hidden line and hidden surface removal problems, the normal step taken in computer graphics methodology is to replace graphical models by *polyhedral models* that give sufficient information for determining the hidden parts of objects. These models are constructed from two-dimensional primitives, polygonal faces, rather than just lines.

Unfortunately, new problems arise. Usually, hidden line removal algorithms assume that the polygons do not intersect each other except at common edges or vertices. Obviously, a properly constructed polyhedral model of a physical shape cannot include intersecting polygons, because the surface of the object would otherwise intersect itself; hence, it is natural to consider only non-selfintersecting polyhedral models valid. But how to ensure that models satisfy criteria for correctness such as this?

The *integrity* of solid models is one of the central issues of this book. Solid models are of little value if the construction of correct models is

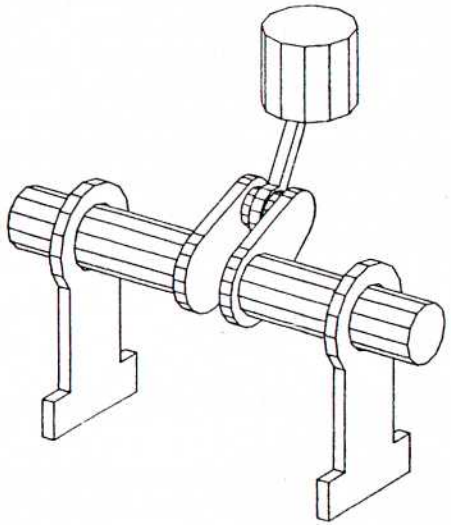

Figure 1.3 A polyhedral model.

excessively complicated. Note that mere integrity *checking* (say, detection of illegal intersections in the hidden line removal example) is only a partial solution to the problem of integrity, because the user must figure out the corrective actions to be taken himself, and a "trial-and-error" sequence is likely to emerge.

Integrity *enforcement* would avoid this problem by making the generation of incorrect models impossible. The new problem of providing sufficient integrity checks without penalizing in the ease of use and the flexibility of the modeling system now arises, however.

1.3.3 Complexity and Geometric Coverage

The problem of integrity is related to another problem, the *complexity* of generating a polyhedral model. Even a relatively simple object such as the one of Figure 1.3 requires more than one hundred polygons. It is a complicated, boresome, and error-prone task to generate such information by hand.

The geometric coverage of a polyhedral model is not sufficient for tasks that require accurate modeling of curved shapes—say, automobile body design. Industries that need such models developed very early methods

1. *What does the object look like?*
2. *What is the weight, surface area, etc. of the object?*
3. *Does this object hit this other one as they move?*
4. *Is the object strong enough to carry this load?*
5. *How can the object be manufactured with certain available manufacturing processes?*

Table 1.1 Geometric questions.

for dealing with quite complex shapes. Unfortunately, the difficulty of dealing with solid geometry in a computer rises sharply with the complexity of mathematical formulations used. Not very many solid modelers offer complete functionality for quadric surfaces and tori, and probably none for freeform surfaces.

1.3.4 Nature of Geometric Computation

According to our desire of general applicability, we expect solid models to be capable of providing *algorithmically* answers to typical geometric questions arising in engineering applications. Examples of such questions are listed in Table 1.1.

Observe that the result of a geometric question may be an image, a single number, or a Boolean constant (*true*, *false*). In fact, it can be another solid model that models the result of the calculation, as with the question "*what is the effect of this manufacturing process applied to this object?*" Clearly, it is important that a geometric modeler includes facilities to model not only physical objects, but also effects of *physical processes* applied to them. Of course, the modeler should be capable of repeated application of these operations to the results of previous similar operations. In other words, operations available in the modeler should constitute a *closed system* where all operations are guaranteed to maintain the correctness of the underlying models.

1.4 SOLID MODELING SYSTEM

One of the underlying ideas of geometric modeling is that it makes sense to separate modeling from the applications, and to seek modeling techniques

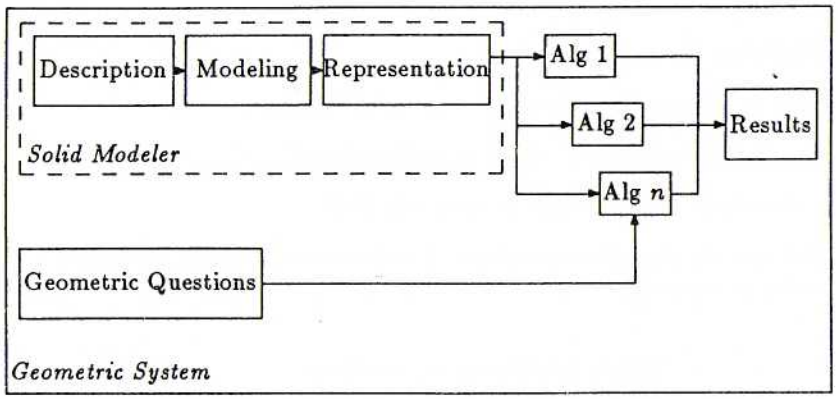

Figure 1.4 Functional components of solid modeler.

that are relatively independent of the particular objects modeled, and of the intended use of the models. Hence, is it possible to functionally distinguish from the application proper a piece of software that deals with solid models and provides answers to geometric questions such as those of Table 1.1—the *solid modeling system* or just *solid modeler* for short.

In addition to the problems of completeness, integrity, complexity, and geometric coverage discussed in the preceding section, there are many practical points that make the construction of a solid modeler a nontrivial task. To see why this is so, let us examine the functional components of a solid modeler and its role in a "geometric system" as depicted in Figure 1.4 [95].

Initially, objects are described for the modeler in terms of a *description language* based on the *modeling concepts* available in the solid modeler. The user may enter the description simply as written text, or preferably through a *user interface* with help of graphical interaction.

Once entered, object descriptions are translated to create the actual *internal representations* stored by the modeler. The relationship between the description language and the internal representations need not be a direct one: internal representations can employ modeling concepts different to the original description. Moreover, a solid modeler may well include several description languages intended for different kinds of users and applications.

The modeler must provide interfaces for communicating with other systems. These interfaces are used to transmit information for various algorithms, or perhaps complete solid models for other design systems. The

modeler must also include facilities for storing object descriptions and other data stored by the modeler into permanent data bases.

A solid modeler must necessarily handle geometric data in different, coexisting representations. The description language, of course, forms a solid model as such. For purposes of calculating answers to geometric questions, a transformation from the external description to an internal representation is necessary. In fact, for efficiency a solid modeler should support multiple internal representations of objects; hence, it must include *conversion algorithms* that can modify data from one representation to another.

PROBLEMS

1.1. List some other kinds of models you are familiar with. Can these models be represented in a computer?

1.2. In Mozart's opera *The Magic Flute*, prince Tamino fells in love with Pamina just by looking at her picture. To which extent is Tamino using a model in a reasonable fashion?

1.3. The extent to which the questions of Table 1.1 can really be considered "geometric" is actually debatable. List the actual information required for answering each of the questions. What information can be expected to be available in a generic geometric modeling system?

BIBLIOGRAPHIC NOTES

The separation of various kinds of geometric models is not strict, and the identical data structures and algorithms may be applicable in several kinds of models. For instance, picture files of graphics systems may consist of exactly identical primitives as geometric models of printed circuit boards, and many algorithms are of equal interest in both cases. Here, the distinction is more a statement of the intent than of the content of the model.

The separation of "modeling" and "graphics" is nevertheless one of the fundamental ideas behind the current standard device-independent graphics software packages such as GKS [57]. The roots of this distinction can be found from a conference held in Seillac, France in 1976 [47]; see also [89].

The design and implementation of "geometric systems" in the sense of Figure 1.4 is beyond the scope of this book. For further discussion on the architectures of computer-aided design systems, see e.g. the book by Encarnação and Schlechtendahl [37] and the proceedings [36].